01NAEX - Lecture 07
The 2^k Factorial Design III:
Center Points, Blocking and Confounding

Jiri Franc

Czech Technical University
Faculty of Nuclear Sciences and Physical Engineering
Department of Mathematics

The 2^k Factorial Design

Last lecture: a single replicate of the 2^k design

- zero degrees of freedom for error, sparsity of effects,
- normal and half-normal plots of the effects, Lenth's method,
- projection, transformation, outliers and dispersion effects.

Today: (Montgomery, Sections 6.7–6.8 and 7.1–7.8)

- why the 2^k designs are optimal designs,
- center points: pure error and a test for curvature,
- blocking a replicated 2^k design,
- confounding the 2^k design in 2, 4, . . . , 2^p blocks,
- partial confounding.

2^k Designs are Optimal Designs

Two-level factorial designs have many interesting properties. One of the most useful is that 2^k designs minimize the variance of the model regression coefficients.

Consider a simple case: the 2^2 design with one replicate and coded variables.

- The regression model fitted to the data is

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 + \varepsilon.$$

- The four runs of this design in terms of the regression model are

$$\begin{aligned}
(1) &= \beta_0 + \beta_1(-1) + \beta_2(-1) + \beta_{12}(-1)(-1) + \varepsilon_1 \\
a &= \beta_0 + \beta_1(1) + \beta_2(-1) + \beta_{12}(1)(-1) + \varepsilon_2 \\
b &= \beta_0 + \beta_1(-1) + \beta_2(1) + \beta_{12}(-1)(1) + \varepsilon_3 \\
ab &= \beta_0 + \beta_1(1) + \beta_2(1) + \beta_{12}(1)(1) + \varepsilon_4
\end{aligned}$$

- In matrix form

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

2^k Designs are Optimal Designs (2)

The regression coefficients are estimated by ordinary least squares (OLS)

$$\hat{\boldsymbol{\beta}}^{(OLS)} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y},$$

where

$$\mathbf{X}'\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 \end{bmatrix}\begin{bmatrix} 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

$$\mathbf{X}'\mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 \end{bmatrix}\begin{bmatrix} (1) \\ a \\ b \\ ab \end{bmatrix} = \begin{bmatrix} (1) + a + b + ab \\ -(1) + a - b + ab \\ -(1) - a + b + ab \\ (1) - a - b + ab \end{bmatrix}$$

The $\mathbf{X}'\mathbf{X}$ matrix is diagonal because the 2^k designs are orthogonal.

2^k Designs are Optimal Designs (3)

In the 2^2 example the least squares estimates are

$$\hat{\boldsymbol{\beta}}^{(OLS)} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y} = \begin{pmatrix} \frac{(1)+a+b+ab}{4} \\ \frac{-(1)+a-b+ab}{4} \\ \frac{-(1)-a+b+ab}{4} \\ \frac{(1)-a-b+ab}{4} \end{pmatrix}$$

The least squares estimates of the regression coefficients other than the intercept are exactly equal to one-half of the usual effect estimates!

The variance of any model regression coefficient is easy to find

$$V(\hat{\beta}_j) = \sigma^2 \left[(\mathbf{X}'\mathbf{X})^{-1}\right]_{jj} = \frac{\sigma^2}{n2^k} = \frac{\sigma^2}{N}.$$

All model regression coefficients have the same variance, and no other design with N runs in the region $-1 \leq x_i \leq 1$ gives a smaller variance of the coefficients.

D-Optimality

Linear model: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon},\ \boldsymbol{\varepsilon} \sim N(\mathbf{0}, \sigma^2\mathbf{I})$. Information matrix: $\mathbf{M} = \mathbf{X}'\mathbf{X}$.

- A **D-optimal** design maximizes $\det(\mathbf{X}'\mathbf{X})$.
- The volume of the joint confidence region for $\boldsymbol{\beta}$ is inversely proportional to $\sqrt{\det(\mathbf{X}'\mathbf{X})}$, so a D-optimal design gives the most precise estimates of the coefficients as a whole.
- For the 2^2 design $\det(\mathbf{X}'\mathbf{X}) = 4^4 = 256$, the maximum possible value for a four-run design in the square $-1 \le x_1, x_2 \le 1$.
- The 2^k design is D-optimal for fitting the first-order model or the first-order model with interactions.
- D-optimal designs for irregular regions or unusual models are found by exchange algorithms (JMP, Design-Expert, Minitab; we will meet them later).
- Adding a center point helps to check curvature, but it lowers the D-efficiency $\det(\mathbf{X}'\mathbf{X})^{1/p}/N$: for the 2^2 design with one center point it is $(5 \cdot 4^3)^{1/4}/5 = 0.85$ instead of 1.

G-Optimality

The variance of the predicted response depends on the point **x** where we predict. For the 2^2 design the estimates are independent with variance $\sigma^2/4$, so

$$\begin{aligned} V[\hat{y}(x_1, x_2)] &= V(\hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_{12} x_1 x_2) \\ &= \frac{\sigma^2}{4}\left(1 + x_1^2 + x_2^2 + x_1^2 x_2^2\right). \end{aligned}$$

- $V[\hat{y}(0,0)] = \sigma^2/4, \quad V[\hat{y}(1,0)] = \sigma^2/2,$ the maximum is at the corners: $V[\hat{y}(\pm 1, \pm 1)] = \sigma^2$.
- In general $V[\hat{y}(\mathbf{x})] = \sigma^2\, \mathbf{f}(\mathbf{x})'(\mathbf{X}'\mathbf{X})^{-1}\mathbf{f}(\mathbf{x})$, where $\mathbf{f}(\mathbf{x})$ is the row of the model matrix at **x**.
- A **G-optimal** design minimizes the maximum prediction variance $\max_{\mathbf{x}} V[\hat{y}(\mathbf{x})]$ over the design region. The smallest attainable maximum is $p\sigma^2/N$ (p = number of parameters).
- For the 2^2 design $p = N = 4$ and the maximum is $\sigma^2 = p\sigma^2/N$: the 2^k design is G-optimal for the first-order model with interactions.

I-Optimality

An **I-optimal** design minimizes the *average* prediction variance over the design region (the integrated variance criterion)

$$
\begin{aligned}
I &= \frac{1}{A}\int_{-1}^{1}\int_{-1}^{1} V[\hat{y}(x_1, x_2)]\, dx_1\, dx_2 \\
&= \frac{1}{4}\int_{-1}^{1}\int_{-1}^{1} \frac{\sigma^2}{4}\left(1 + x_1^2 + x_2^2 + x_1^2 x_2^2\right) dx_1\, dx_2 = \frac{4\sigma^2}{9},
\end{aligned}
$$

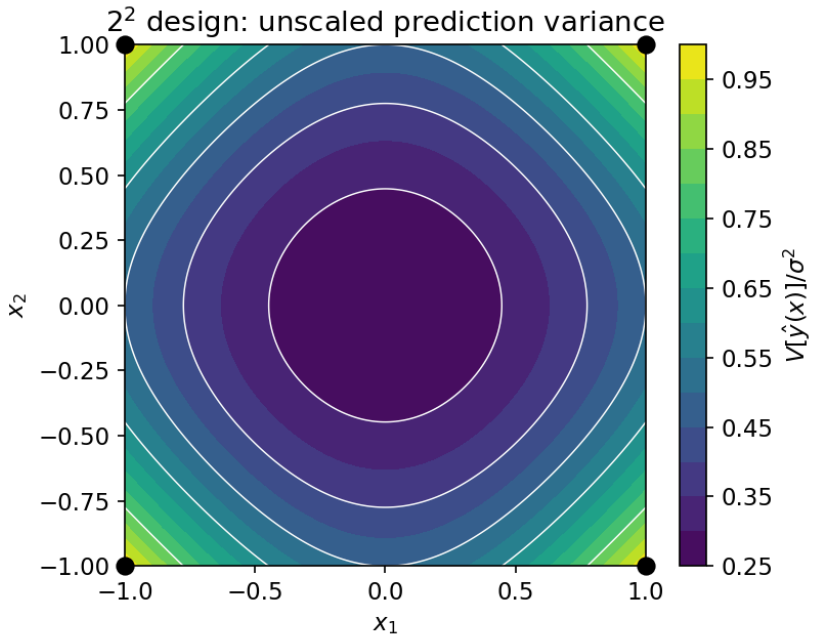
where $A = 4$ is the area (in general the volume) of the design region.

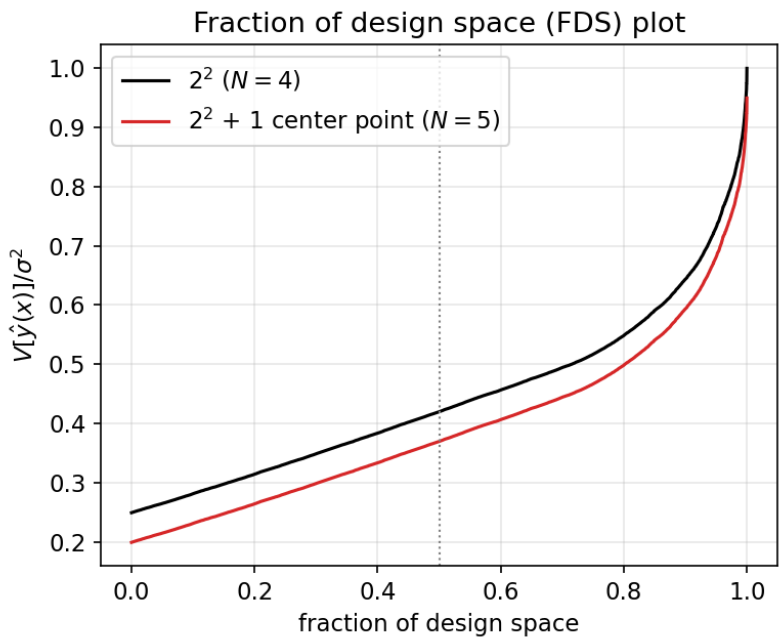
- $4\sigma^2/9$ is the smallest average prediction variance of any four-run design for the first-order model with interaction in this region: the 2^k design is also I-optimal.
- I-optimal designs are useful when prediction of the response is the goal of the experiment.

Heuristic: D for precise β, I or G for precise $\hat{y}$ (I = average, G = worst case). The design region should match the process window we care about.

Prediction Variance of the 2^2 Design in Python

Left: the unscaled prediction variance $V[\hat{y}(\mathbf{x})]/\sigma^2$ of the 2^2 design. The contours are nearly circular and the variance is largest at the corners.
Right: the **fraction of design space** (FDS) plot (Montgomery, Figure 6.36): the prediction variance is at most $0.42\,\sigma^2$ on 50 % of the design region. A flat and low FDS curve is ideal. One center point lowers the unscaled variance everywhere, but it costs one more run.





Addition of Center Points to a 2^k Design

In a two-level factorial design we assume linearity in the factor effects. Perfect linearity is not necessary: adding interaction terms to the first-order model already allows some curvature (twisting of the plane).

First-order model with interactions

$$y = \beta_0 + \sum_{j=1}^{k} \beta_j x_j + \sum_{i<j} \beta_{ij} x_i x_j + \varepsilon$$

Second-order response surface model

$$y = \beta_0 + \sum_{j=1}^{k} \beta_j x_j + \sum_{i<j} \beta_{ij} x_i x_j + \sum_{j=1}^{k} \beta_{jj} x_j^2 + \varepsilon$$

where β_{jj} are pure quadratic effects.

Addition of center points

- Based on the idea of replicating some of the runs in a factorial design.
- Runs at the center provide an estimate of error and allow the experimenter to distinguish between the two models.

Addition of Center Points to a 2^k Design

The method adds n_C replicated runs at the center of the 2^k design; this provides protection against curvature from second-order effects and allows an independent estimate of error.

- The center point consists of n_C replicates run at $x_1 = x_2 = \ldots = x_k = 0$.
- The k factors have to be **quantitative**.
- Center points do not affect the usual effect estimates of the 2^k design (all contrast columns are 0 at the center).

Test for Pure Quadratic Curvature

Compute

- $\bar{y}_F$ – the average of the n_F runs at the factorial points,
- $\bar{y}_C$ – the average of the n_C runs at the center point,
- $\bar{y}_F - \bar{y}_C$ – the difference between the two averages.

If the difference $\bar{y}_F - \bar{y}_C$ is small, then the center points lie on or near the (hyper)plane passing through the factorial points; more precisely, the model with main effects and interactions predicts $\bar{y}_F$ at the center. If it is large, then quadratic curvature is present.

A single-degree-of-freedom **sum of squares for pure quadratic curvature**

$$SS_{Pure\ quadratic} = \frac{n_F n_C (\bar{y}_F - \bar{y}_C)^2}{n_F + n_C}$$

is compared with the pure error mean square $MS_E = \sum_{\text{center}} (y_i - \bar{y}_C)^2 / (n_C - 1)$. It tests $H_0 : \sum_{j=1}^{k} \beta_{jj} = 0$ vs $H_1 : \sum_{j=1}^{k} \beta_{jj} \neq 0$.

Equivalently, a t-test: $t_0 = (\bar{y}_F - \bar{y}_C) / \sqrt{MS_E\,(1/n_F + 1/n_C)}$ with $n_C - 1$ degrees of freedom, $t_0^2 = F_0$.

We can only test the *sum* of the quadratic effects: all columns x_j^2 are equal (1 at the factorial points, 0 at the center).

Center Points in the Filtration Rate Experiment (Example 6.7)

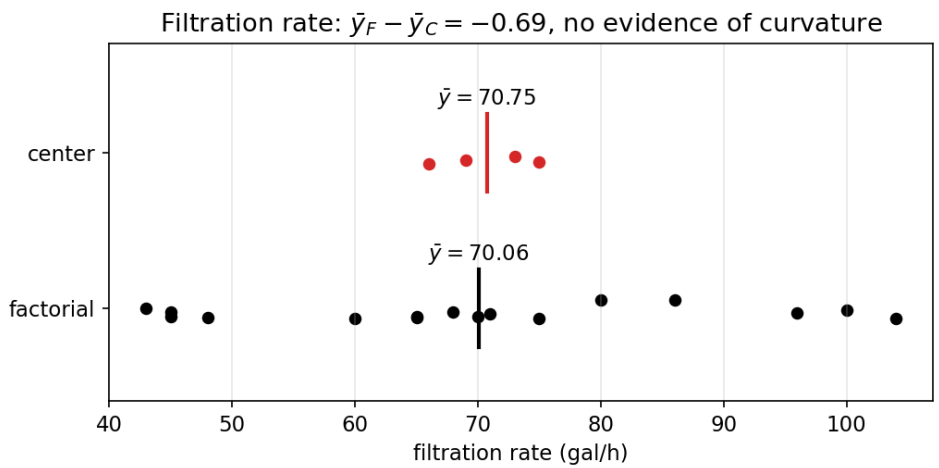
Recall the unreplicated 2^4 design of Example 6.2 (filtration rate; A temperature, B pressure, C concentration of formaldehyde, D stirring rate). Suppose that four center points were added, with filtration rates 73, 75, 66 and 69.

$\bar{y}_C = 70.75$ and $\bar{y}_F = 70.06$ are very similar, so we suspect that there is no strong curvature.

$$MS_E = \frac{\sum_{i=1}^{4}(y_i - \bar{y}_C)^2}{n_C - 1} = \frac{\sum_{i=1}^{4}(y_i - 70.75)^2}{4 - 1} = \frac{48.75}{3} = 16.25,$$

$$SS_{\text{Pure quadratic}} = \frac{n_F n_C (\bar{y}_F - \bar{y}_C)^2}{n_F + n_C} = \frac{(16)(4)(-0.69)^2}{16 + 4} = 1.51,$$

$F_0 = 1.51/16.25 = 0.093$ with 1 and 3 degrees of freedom, $p = 0.780$. No evidence of second-order curvature.



Test for Curvature in Python

The data with the four center points (coded 0) are in the file `data/Pilot_Plant_Filtration_Rate_Experiment3.csv`.

```
fname = "Pilot_Plant_Filtration_Rate_Experiment3.csv"
f = pd.read_csv(URL + fname, sep=";")       # 16 + 4 center points
f["cp"] = (f["A"] == 0).astype(int)         # 1 = center point
yF = f.loc[f.cp == 0, "Rate"].mean()         # 16 factorial runs
yC = f.loc[f.cp == 1, "Rate"].mean()         # 4 center points
nF, nC = 16, 4
SS_pq = nF * nC * (yF - yC)**2 / (nF + nC)
MS_pe = f.loc[f.cp == 1, "Rate"].var()       # pure error, 3 df
F0 = SS_pq / MS_pe
print(f"yF = {yF:.4f}, yC = {yC:.2f}, SS_pq = {SS_pq:.4f}")
print(f"MS_pe = {MS_pe:.2f}, F0 = {F0:.3f}, "
      f"p = {stats.f.sf(F0, 1, nC - 1):.4f}")
```

```
yF = 70.0625, yC = 70.75, SS_pq = 1.5125
MS_pe = 16.25, F0 = 0.093, p = 0.7802
```

Full Model with the Center-Point Indicator

The indicator `cp` carries the curvature sum of squares, the residual is the pure error (Montgomery, Table 6.24). Significant: *A*, *C*, *D*, *AC*, *AD* – as in Lecture 6, but 3 df for error give low power.

```
full = smf.ols("Rate ~ A*B*C*D + cp", data=f).fit()
print(anova_lm(full).round(3))
```

	df	sum_sq	mean_sq	F	PR(>F)
A	1.0	1870.563	1870.563	115.112	0.002
B	1.0	39.063	39.063	2.404	0.219
A:B	1.0	0.062	0.062	0.004	0.954
C	1.0	390.063	390.063	24.004	0.016
A:C	1.0	1314.063	1314.063	80.865	0.003
B:C	1.0	22.562	22.562	1.388	0.324
A:B:C	1.0	14.063	14.063	0.865	0.421
D	1.0	855.563	855.563	52.650	0.005
A:D	1.0	1105.562	1105.562	68.035	0.004
B:D	1.0	0.563	0.563	0.035	0.864
A:B:D	1.0	68.063	68.063	4.188	0.133
C:D	1.0	5.062	5.062	0.312	0.616
A:C:D	1.0	10.562	10.562	0.650	0.479
B:C:D	1.0	27.562	27.562	1.696	0.284
A:B:C:D	1.0	7.562	7.562	0.465	0.544
cp	1.0	1.513	1.513	0.093	0.780
Residual	3.0	48.750	16.250	NaN	NaN

Reduced Model, Curvature and Lack of Fit

```
red = smf.ols("Rate ~ A*C + A*D + cp", data=f).fit()
print(anova_lm(red).round(3))
print(anova_lm(red, full).round(3))  # lack of fit vs pure error
```

```
             df     sum_sq   mean_sq       F  PR(>F)
A           1.0  1870.563  1870.563  99.712   0.000
C           1.0   390.063   390.063  20.793   0.001
A:C         1.0  1314.063  1314.063  70.047   0.000
D           1.0   855.563   855.563  45.607   0.000
A:D         1.0  1105.562  1105.562  58.933   0.000
cp          1.0     1.513     1.513   0.081   0.781
Residual   13.0   243.875    18.760     NaN     NaN
   df_resid      ssr  df_diff  ss_diff      F  Pr(>F)
0      13.0  243.875      0.0      NaN    NaN     NaN
1       3.0   48.750     10.0  195.125  1.201   0.494
```

The residual of the reduced model splits into **lack of fit** (195.125, 10 df) and **pure error** (48.75, 3 df); $F = 1.20$, $p = 0.494$: no significant lack of fit (only 3 df for pure error, so the test has low power).

Without the indicator `cp` the residual of `Rate ~ A*C + A*D` fitted to all 20 runs has 14 df:

$$SS_{\text{Pure quadratic}} + SS_{\text{Lack of fit}} + SS_{\text{Pure error}} = 1.51 + 195.13 + 48.75 = 245.39.$$

Curvature as a Regression Coefficient

The coefficient of the center-point indicator is $\bar{y}_C - \bar{y}_F$; its t-test uses the residual of the reduced model (13 df), so $p = 0.781$ differs slightly from 0.780 (pure error, 3 df).

```
print(pd.DataFrame({"coef": red.params, "se": red.bse,
                    "t": red.tvalues, "p": red.pvalues}).round(4))
q = smf.ols("Rate ~ A*C + A*D + I(A**2)", data=f).fit()
b, p = q.params["I(A ** 2)"], q.pvalues["I(A ** 2)"]
print(f"I(A**2): coef {b:.4f}, p {p:.4f}")
```

```
               coef      se        t       p
Intercept   70.0625  1.0828  64.7044  0.0000
A           10.8125  1.0828   9.9856  0.0000
C            4.9375  1.0828   4.5599  0.0005
A:C         -9.0625  1.0828  -8.3694  0.0000
D            7.3125  1.0828   6.7533  0.0000
A:D          8.3125  1.0828   7.6768  0.0000
cp           0.6875  2.4212   0.2839  0.7809
I(A**2): coef -0.6875, p 0.7809
```

Any term x_j^2, e.g. `I(A**2)`, gives the same test with the opposite sign: the columns $x_1^2 = \cdots = x_k^2$ are identical, so only $\sum_j \beta_{jj}$ is estimable. To estimate the individual β_{jj} we need more design points.

When the Curvature Test is Significant

- The second-order model for $k = 2$, $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2 + \varepsilon$, has six parameters, but the 2^2 design with center points has only five distinct points.
- A simple and very effective solution: augment the 2^k design with $2k$ **axial runs** $(\pm\alpha, 0, \ldots, 0), \ldots, (0, \ldots, 0, \pm\alpha)$. The result is the **central composite design** (CCD): $2^k + 2k + n_C$ runs.
- For $k = 3$ the CCD has $14 + n_C$ runs (usually $3 \leq n_C \leq 5$) for the ten-parameter second-order model.
- The factorial part with center points is run first; the axial runs are added only if curvature is detected – **sequential experimentation**.
- Central composite designs and response surface methodology: Montgomery, Chapter 11.

Center Points – Practical Remarks (Montgomery, Section 6.8)

1. In an ongoing process, use the **current operating conditions** as the center point. The operating personnel get familiar runs, and the results are unlikely to be worse than usual.
2. The center-point responses can be compared with the historical process performance (e.g. a control chart): a rough check whether anything unusual happened during the experiment.
3. Run the center points in **nonrandom order**, spread in time (beginning, middle, end of the experiment). Plotted against time, they reveal a trend or instability of the process.
4. With little prior information about the process variability, run two or three center points **first**. If the variability is much larger than expected, stop and find out why.
5. With a **qualitative factor** (e.g. catalyst type) the center points are run at the center of the quantitative factors, at each level of the qualitative factor.

Blocking in the 2^k Factorial Design

Sometimes it is not feasible or practical to run a completely randomized experiment, e.g. one batch of raw material is not large enough for all runs, or we want the conclusions to hold across several batches, days or operators.

- Blocking is a technique for dealing with controllable nuisance variables (factors that probably affect the response, but we are not interested in their effect).
- Two situations:
  - **replicated design** – each replicate is a block (time period, batch of raw material, . . .), runs within a block are randomized;
  - **block smaller than one replicate** – some effects must be **confounded** with blocks.
- The analysis of a replicated design in blocks is the RCBD analysis from Lectures 3 and 4: blocks $+$ factorial effects.

Blocking a Replicated 2^k Design (Example 7.1)

Chemical process example from Lecture 5: A – reactant concentration, B – amount of catalyst, y – recovery.
Nuisance variable: raw material. Only four trials can be run from one batch, so each replicate is run with a different batch.

	Factor		Treatment	Replicate			
	A	B	Combination	I	II	III	Total
(1)	−	−	A low, B low	28	25	27	80
a	+	−	A high, B low	36	32	32	100
b	−	+	A low, B high	18	19	23	60
ab	+	+	A high, B high	31	30	29	90

Blocking a Replicated 2^k Design (Example 7.1)

2 factors, 3 replicates = 3 blocks (Montgomery, Tables 7.1 and 7.2)

	Block 1	Block 2	Block 3
	$(1) = 28$ $a = 36$ $b = 18$ $ab = 31$	$(1) = 25$ $a = 32$ $b = 19$ $ab = 30$	$(1) = 27$ $a = 32$ $b = 23$ $ab = 29$
Block totals:	$B_1 = 113$	$B_2 = 106$	$B_3 = 111$

Source of Variation	Sum of Squares	Degrees of Freedom	Mean Square	F_0	P-Value
Blocks	6.50	2	3.25		
A (concentration)	208.33	1	208.33	50.32	0.0004
B (catalyst)	75.00	1	75.00	18.12	0.0053
AB	8.33	1	8.33	2.01	0.2060
Error	24.84	6	4.14		
Total	323.00	11			

$SS_{Blocks} = \sum_i B_i^2/4 - y_{..}^2/12 = 6.50$ with 2 df. The conclusions are the same as without blocks; the block effect is small.

Example 7.1 in Python

```
ch = pd.DataFrame({
    "A": [-1, 1, -1, 1] * 3, "B": [-1, -1, 1, 1] * 3,
    "Block": pd.Categorical(np.repeat([1, 2, 3], 4)),
    "y": [28, 36, 18, 31, 25, 32, 19, 30, 27, 32, 23, 29]})
blocked = smf.ols("y ~ Block + A*B", data=ch).fit()
print(anova_lm(blocked).round(4))
```

```
            df    sum_sq   mean_sq         F  PR(>F)
Block      2.0    6.5000    3.2500    0.7852  0.4978
A          1.0  208.3333  208.3333   50.3356  0.0004
B          1.0   75.0000   75.0000   18.1208  0.0053
A:B        1.0    8.3333    8.3333    2.0134  0.2057
Residual   6.0   24.8333    4.1389       NaN     NaN
```

- The factorial sums of squares are the same as in the unblocked analysis; the block sum of squares is taken out of the error.
- Here $MS_{Blocks} = 3.25 < MS_E$: blocking did not help much (without blocks $MS_E = 3.92$ with 8 df). We still had to block – we did not know it in advance.
- The F-test for blocks is not a proper test (blocks are a restriction on randomization, Lecture 3); compare MS_{Blocks} with MS_E only informally.

Confounding in the 2^k Factorial Design

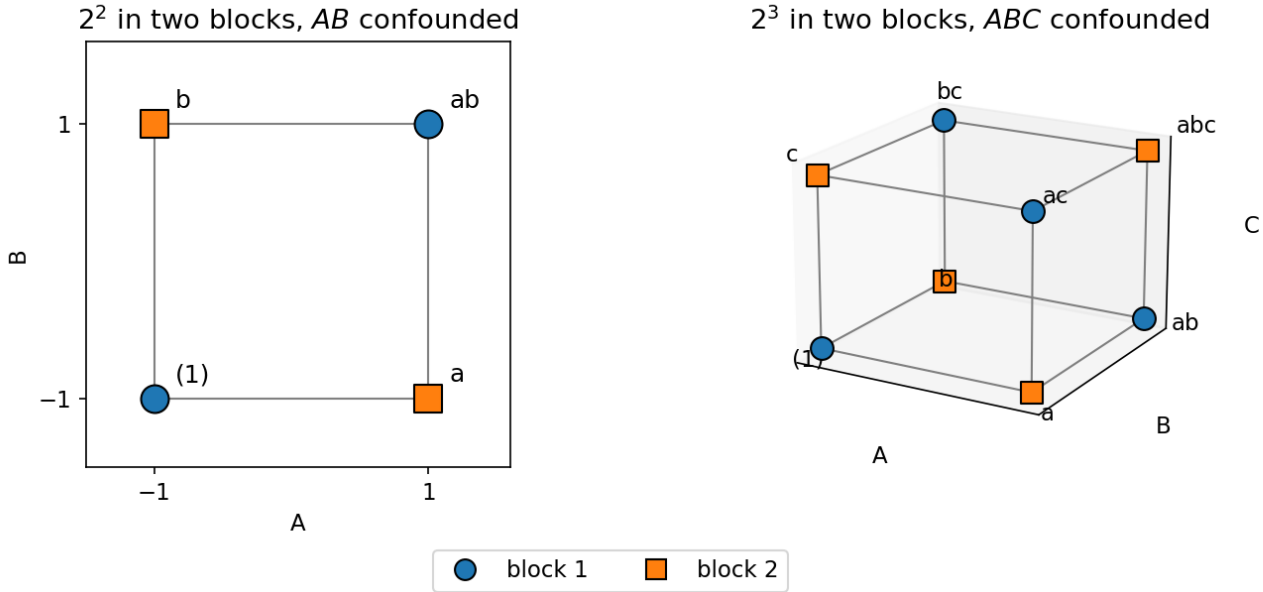
What if the block size is smaller than the number of treatment combinations in one replicate? E.g. a single replicate of the 2^5 design needs 32 runs, but one batch of raw material is enough for only 16 or 8 runs.

- **Confounding** arranges a complete factorial in incomplete blocks. Information about certain effects (usually high-order interactions) becomes indistinguishable from (confounded with) blocks.
- The 2^k design is run in 2^p blocks of 2^{k-p} runs, $p < k$: two blocks ($p = 1$), four blocks ($p = 2$), eight blocks ($p = 3$), ...

How do we confound into two blocks?

- Choose the effect to be confounded, usually the highest-order interaction.
- Assign all runs with $+$ in its column to one block and all runs with $-$ to the other block.
- Randomize the order of the blocks and the order of the runs within each block.

The 2^2 and 2^3 Designs in Two Blocks



- 2^2: block 1 $= \{(1), ab\}$, block 2 $= \{a, b\}$. $A = \frac{1}{2}[ab + a - b - (1)]$ has one plus and one minus run from each block, so a block difference cancels. Same for B.
- $AB = \frac{1}{2}[ab + (1) - a - b]$: both plus runs are in block 1, both minus runs in block 2 – AB is **identical** with the block difference: AB is **confounded** with blocks.
- 2^3 with ABC confounded: block 1 $= \{(1), ab, ac, bc\}$ ($ABC = -$), block 2 $= \{a, b, c, abc\}$ ($ABC = +$). All main effects and two-factor interactions are free of blocks.

Constructing the Blocks: Defining Contrast and Principal Block

Defining contrast. For the effect $A^{\alpha_1} B^{\alpha_2} \cdots K^{\alpha_k}$ ($\alpha_i \in \{0, 1\}$) define

$$L = \alpha_1 x_1 + \alpha_2 x_2 + \cdots + \alpha_k x_k \pmod 2, \qquad x_i \in \{0, 1\} \text{ (low/high level)}.$$

Runs with the same value of L go to the same block. For ABC: $L = x_1 + x_2 + x_3$,

$$\begin{array}{llll} (1): \ L = 0, & a: \ L = 1, & b: \ L = 1, & ab: \ L = 2 \equiv 0, \\ c: \ L = 1, & ac: \ L = 2 \equiv 0, & bc: \ L = 2 \equiv 0, & abc: \ L = 3 \equiv 1. \end{array}$$

Principal block = the block containing (1). Its runs form a group under multiplication modulo 2:

$$ab \cdot ac = a^2 bc = bc, \qquad ab \cdot bc = ab^2 c = ac, \qquad ac \cdot bc = abc^2 = ab.$$

Every other block is obtained by multiplying the principal block by one run not in it, e.g. by b:

$$b \cdot (1) = b, \quad b \cdot ab = a, \quad b \cdot ac = abc, \quad b \cdot bc = c.$$

Constructing the Blocks in Python

```
d = pd.DataFrame(itertools.product([-1, 1], repeat=3),
                 columns=["C", "B", "A"])[["A", "B", "C"]]
d["tc"] = ["".join(c for c in "abc" if r[c.upper()] > 0) or "(1)"
           for _, r in d.iterrows()]
d["ABC"] = d.A * d.B * d.C
d["Block"] = np.where(d.ABC < 0, 1, 2)
print(d.T)
# defining contrast L = x1 + x2 + x3 (mod 2), x_i in {0, 1}
L = ((d[["A", "B", "C"]] + 1) // 2).sum(axis=1) % 2
print(L.tolist())
```

```
          0   1   2   3   4   5   6    7
A        -1   1  -1   1  -1   1  -1    1
B        -1  -1   1   1  -1  -1   1    1
C        -1  -1  -1  -1   1   1   1    1
tc      (1)   a   b  ab   c  ac  bc  abc
ABC      -1   1   1  -1   1  -1  -1    1
Block     1   2   2   1   2   1   1    2
[0, 1, 1, 0, 1, 0, 0, 1]
```

Both methods give the same blocks: $L = 0 \Leftrightarrow ABC = -1$ for the 2^3 design (in general $L = 0$ corresponds to the sign $(-1)^{\text{order}}$ of the effect). The run order within each block is then randomized.

Estimation of Error in a Confounded Design

- For small k (2 or 3) the confounded design must be **replicated** to estimate the error. Example: the 2^3 design in two blocks with ABC confounded, replicated four times = 8 blocks, 32 runs (Montgomery, Table 7.5):

Source of variation	df
Replicates	3
Blocks (ABC)	1
Error for ABC (replicates $\times$ blocks)	3
A, B, C, AB, AC, BC	1 each
Error (replicates $\times$ effects)	18
Total	31

- ABC is confounded in every replicate (**complete confounding**): no information on ABC. Better: confound a different effect in each replicate – **partial confounding** (later today).
- For $k \geq 4$ we can often afford only one replicate: the high-order interactions are pooled as error, or we use the normal plot of effects and Lenth's method (Lecture 6).

Filtration Rate Experiment in Two Blocks (Example 7.2)

Number	A	B	C	D	Run Label	(gal/h)
1	—	−	−	−	(1)	45
2	+	−	−	−	a	71
3	−	+	−	−	b	48
4	+	+	−	−	ab	65
5	−	−	+	−	c	68
6	+	−	+	−	ac	60
7	−	+	+	−	bc	80
8	+	+	+	−	abc	65
9	−	−	−	+	d	43
10	+	−	−	+	ad	100
11	−	+	−	+	bd	45
12	+	+	−	+	abd	104
13	−	−	+	+	cd	75
14	+	−	+	+	acd	86
15	−	+	+	+	bcd	70
16	+	+	+	+	$abcd$	96

Suppose that only 8 runs can be made from one batch of raw material: the 2^4 design must be run in two blocks of 8 runs. It is logical to confound the highest-order interaction $ABCD$ with blocks, $L = x_1 + x_2 + x_3 + x_4 \pmod 2$.

Filtration Rate Experiment in Two Blocks (Example 7.2)

	A	B	AB	C	AC	BC	ABC	D	AD	BD	ABD	CD	ACD	BCD	$ABCD$
(1)	−	−	+	−	+	+	−	−	+	+	−	+	−	−	+
a	+	−	−	−	−	+	+	−	−	+	+	+	+	−	−
b	−	+	−	−	+	−	+	−	+	−	+	+	−	+	−
ab	+	+	+	−	−	−	−	−	−	−	−	+	+	+	+
c	−	−	+	+	−	−	+	−	+	+	−	−	+	+	−
ac	+	−	−	+	+	−	−	−	−	+	+	−	−	+	+
bc	−	+	−	+	−	+	−	−	+	−	+	−	+	−	+
abc	+	+	+	+	+	+	+	−	−	−	−	−	−	−	−
d	−	−	+	−	+	+	−	+	−	−	+	−	+	+	−
ad	+	−	−	−	−	+	+	+	+	−	−	−	−	+	+
bd	−	+	−	−	+	−	+	+	−	+	−	−	+	−	+
abd	+	+	+	−	−	−	−	+	+	+	+	−	−	−	−
cd	−	−	+	+	−	−	+	+	−	−	+	+	−	−	+
acd	+	−	−	+	+	−	−	+	+	−	−	+	+	−	−
bcd	−	+	−	+	−	+	−	+	−	+	−	+	−	+	−
$abcd$	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+

Runs with $ABCD = +$ form block 1, runs with $ABCD = -$ block 2.

Suppose that one batch is of much poorer quality: all responses in block 1 are 20 units lower ($(1) = 45 - 20 = 25, \ldots$). The block effect is a *simulated* nuisance effect.

Example 7.2 in Python

```
fname = "Pilot_Plant_Filtration_Rate_Experiment.csv"
g = pd.read_csv(URL + fname, sep=";")
g["Block"] = np.where(g.A * g.B * g.C * g.D > 0, 1, 2)    # ABCD
g["y"] = g.Rate - 20 * (g.Block == 1)     # poor batch: -20
print(g.groupby("Block")["y"].agg(["sum", "mean"]))
g["Block"] = pd.Categorical(g.Block)  # C(Block) fails: C is data
m72 = smf.ols("y ~ Block + A*C + A*D", data=g).fit()
print(anova_lm(m72).round(4))
```

```
       sum    mean
Block
1      406  50.750
2      555  69.375
            df     sum_sq    mean_sq         F  PR(>F)
Block      1.0  1387.5625  1387.5625   66.5808  0.0000
A          1.0  1870.5625  1870.5625   89.7571  0.0000
C          1.0   390.0625   390.0625   18.7168  0.0019
A:C        1.0  1314.0625  1314.0625   63.0540  0.0000
D          1.0   855.5625   855.5625   41.0533  0.0001
A:D        1.0  1105.5625  1105.5625   53.0493  0.0000
Residual   9.0   187.5625    20.8403       NaN     NaN
```

Example 7.2 – Interpretation

- All effects except *ABCD* are **identical** to the effects of Example 6.2: $A = 21.625$, $C = 9.875$, $D = 14.625$, $AC = -18.125$, $AD = 16.625$, ... The normal plot again picks *A*, *C*, *D*, *AC*, *AD*.
- The *ABCD* column now estimates the original interaction plus the block effect:

$$\widehat{ABCD} = \bar{y}_{\text{block 1}} - \bar{y}_{\text{block 2}} = \frac{406}{8} - \frac{555}{8} = -18.625 = 1.375 + (-20).$$

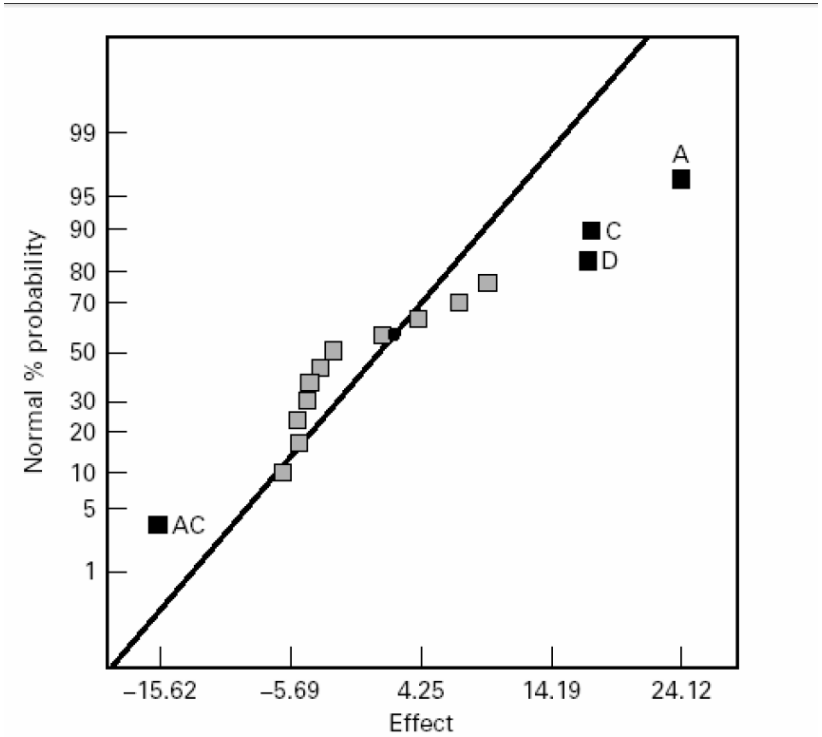
- $SS_{Blocks} = \frac{406^2+555^2}{8} - \frac{961^2}{16} = 1387.56 = SS_{ABCD}$; it is **not** part of the error term.
- The ANOVA of the reduced model with blocks (Montgomery, Table 7.7) gives the same conclusions as in Lecture 6.
- **Python trap:** the formula `C(Block)` fails here, because the data frame has a column `C` that hides the patsy function `C()`. Make the block a categorical column instead.

Why Blocking is Important (Section 7.5)

Now suppose we **did not block**: the 16 runs were made in random order and the nuisance effect −20 hit the first eight runs (in run order).

Run Order	Std. Order	Factor *A*: Temperature	Factor *B*: Pressure	Factor *C*: Concentration	Factor *D*: Stirring Rate	Response Filtration Rate
8	1	−1	−1	−1	−1	25
11	2	1	−1	−1	−1	71
1	3	−1	1	−1	−1	28
3	4	1	1	−1	−1	45
9	5	−1	−1	1	−1	68
12	6	1	−1	1	−1	60
2	7	−1	1	1	−1	60
13	8	1	1	1	−1	65
7	9	−1	−1	−1	1	23
6	10	1	−1	−1	1	80
16	11	−1	1	−1	1	45
5	12	1	1	−1	1	84
14	13	−1	−1	1	1	75
15	14	1	−1	1	1	86
10	15	−1	1	1	1	70
4	16	1	1	1	1	76

Why Blocking is Important (Section 7.5)

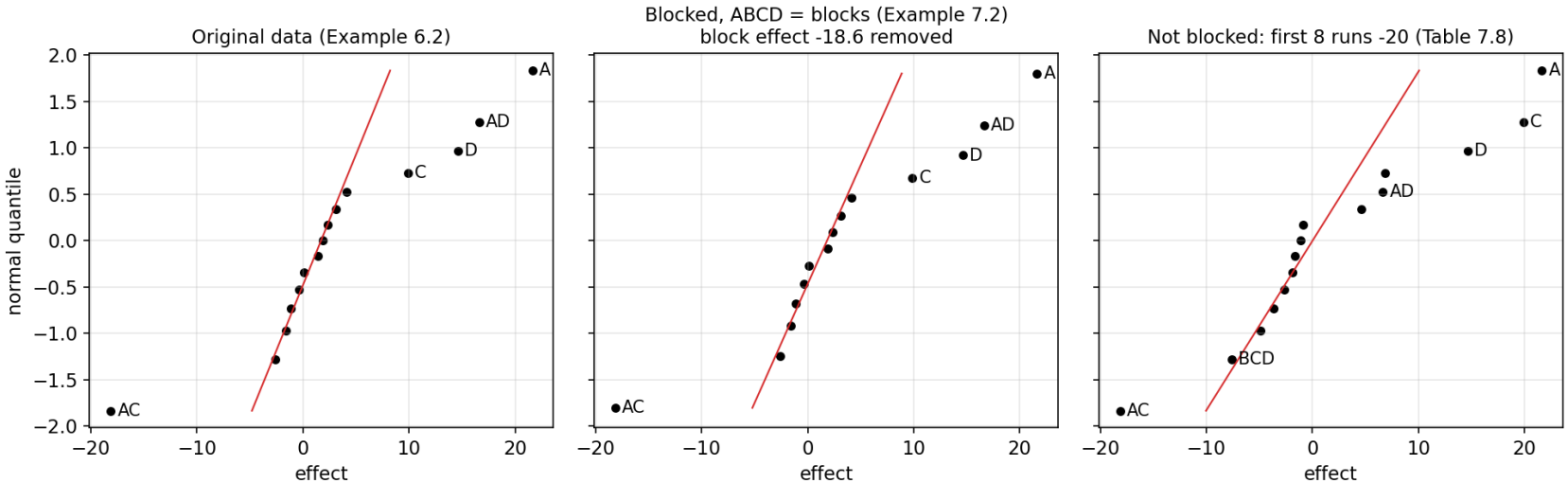


Normal probability plot of the effects of the modified data (Montgomery, Figure 7.5).

- A, C, D and AC appear active, but the important interaction AD is **not identified** (its estimate dropped from 16.6 to 6.6), and C increased from 9.9 to 19.9.
- The nuisance variability is distributed across the other effects, and part of it ends in the error: $MS_E = 81.0$ for $A + C + D + AC$; even with AD added it is 71.5, compared with 19.5 for the original data with the same model $A + C + D + AC + AD$.

Failing to block when we should have can mask real factor effects and distort the interpretation of the experiment.

Blocked vs Not Blocked – Normal Plots in Python



- Left and middle: the same effects – blocking with *ABCD* confounded completely removes the block effect.
- Right: the nuisance effect is mixed into all effects whose sign column is correlated with the indicator of the first eight runs: *AD* falls into the noise, *C* doubles, *BCD* appears.

Confounding the 2^k Design in Four Blocks

Example: the 2^5 design in four blocks of 8 runs. Choose two effects, e.g. *ADE* and *BCE*, with defining contrasts $L_1 = x_1 + x_4 + x_5$, $L_2 = x_2 + x_3 + x_5 \pmod 2$. Runs with the same pair (L_1, L_2) form a block.

```
blocks = {}
for x in itertools.product([0, 1], repeat=5):   # x1, ..., x5
    tc = "".join(c for c, v in zip("abcde", x) if v) or "(1)"
    L1 = (x[0] + x[3] + x[4]) % 2               # ADE
    L2 = (x[1] + x[2] + x[4]) % 2               # BCE
    blocks.setdefault((L1, L2), []).append(tc)
for key, tcs in blocks.items():
    print(key, " ".join(tcs))
```

```
(0, 0) (1) cde bde bc ad ace abe abcd
(1, 1) e cd bd bce ade ac ab abcde
(1, 0) d ce be bcd a acde abde abc
(0, 1) de c b bcde ae acd abd abce
```

Four blocks have 3 df, but *ADE* and *BCE* have only one each: the **generalized interaction** $(ADE)(BCE) = ABCDE^2 = ABCD$ is confounded too. The principal block is again a group: $ad \cdot bc = abcd$, $abe \cdot bde = ad$.

Choosing the Effects Confounded with Blocks

- For four blocks select two effects; their generalized interaction is confounded automatically.
- Care is needed: in the 2^5 design, confounding $ABCDE$ and ABD looks natural (high-order interactions), but it confounds $(ABCDE)(ABD) = A^2B^2CD^2E = CE$, a two-factor interaction that is probably of interest.
- A better choice: ADE and $BCE \Rightarrow ABCD$. It is preferable to sacrifice two three-factor interactions and one four-factor interaction instead of the two-factor interaction CE.
- Check: the product of the signs of ADE and BCE in each block equals the sign of $ABCD$ in that block.

Analysis. The sums of squares of all effects are computed as if no blocking had occurred. The block sum of squares is the sum of the sums of squares of all effects confounded with blocks: $SS_{Blocks} = SS_{ADE} + SS_{BCE} + SS_{ABCD}$ with 3 df.

Confounding the 2^k Design in 2^p Blocks

Two blocks (size 2^{k-1}): one *block generator* G, usually the highest-order interaction. The block effect is aliased with G; all other effects are orthogonal to G and stay unconfounded with blocks.

Four blocks (size 2^{k-2}): two independent generators G_1, G_2; blocks are the four sign combinations of (G_1, G_2), and $G_1 G_2$ is confounded too.

2^p **blocks** (size 2^{k-p}, $p < k$):

- choose p **independent** effects (none of them is a generalized interaction of the others),
- the blocks are given by the p defining contrasts $L_1, \ldots, L_p$,
- exactly $2^p - p - 1$ other effects are confounded: all generalized interactions of the chosen ones; in total $2^p - 1$ effects = df between the blocks.

Example: 2^6 in 8 blocks with *ABEF*, *ABCD*, *ACE* also confounds *CDEF*, *BCF*, *BDE* and *ADF* (Montgomery, Section 7.7).

Suggested Blocking Arrangements for the 2^k Factorial Design

Montgomery, Table 7.9:

Number of Factors, k	Number of Blocks, 2^p	Block Size, 2^{k-p}	Effects Chosen to Generate the Blocks	Interactions Confounded with Blocks
3	2	4	*ABC*	*ABC*
	4	2	*AB, AC*	*AB, AC, BC*
4	2	8	*ABCD*	*ABCD*
	4	4	*ABC, ACD*	*ABC, ACD, BD*
	8	2	*AB, BC, CD*	*AB, BC, CD, AC, BD, AD, ABCD*
5	2	16	*ABCDE*	*ABCDE*
	4	8	*ABC, CDE*	*ABC, CDE, ABDE*
	8	4	*ABE, BCE, CDE*	*ABE, BCE, CDE, AC, ABCD, BD, ADE*
	16	2	*AB, AC, CD, DE*	All two- and four-factor interactions (15 effects)
6	2	32	*ABCDEF*	*ABCDEF*
	4	16	*ABCF, CDEF*	*ABCF, CDEF, ABDE*
	8	8	*ABEF, ABCD, ACE*	*ABEF, ABCD, ACE, BCF, BDE, CDEF, ADF*
	16	4	*ABF, ACF, BDF, DEF*	*ABF, ACF, BDF, DEF, BC, ABCD, ABDE, AD, ACDE, CE, CDF, BCDEF, ABCEF, AEF, BE*
	32	2	*AB, BC, CD, DE, EF*	All two-, four-, and six-factor interactions (31 effects)
7	2	64	*ABCDEFG*	*ABCDEFG*
	4	32	*ABCFG, CDEFG*	*ABCFG, CDEFG, ABDE*
	8	16	*ABCD, CDEF, ADFG*	*ABC, DEF, AFG, ABCDEF, BCFG, ADEG, BCDEG*
	16	8	*ABCD, EFG, CDE, ADG*	*ABCD, EFG, CDE, ADG, ABCDEFG, ABE, BCG, CDFG, ADEF, ACEG, ABFG, BCEF, BDEG, ACF, BDF*

Partial Confounding

- In the replicated design with *ABC* confounded in every replicate (complete confounding), no information on *ABC* can be retrieved.
- **Partial confounding:** confound a **different** effect in each replicate. E.g. four replicates of the 2^3 design in two blocks: *ABC* in replicate I, *AB* in II, *BC* in III, *AC* in IV.
- Each interaction is estimated from the replicates in which it is **not** confounded: *ABC* from II, III, IV; *AB* from I, III, IV; . . . The relative information on these effects is $3/4$ (Yates, 1937).
- ANOVA (Montgomery, Table 7.10):

Source of variation	df
Replicates	3
Blocks within replicates [*ABC* (I) + *AB* (II) + *BC* (III) + *AC* (IV)]	4
A, *B*, *C*	1 each
AB (from I, III, IV), *AC* (I, II, III), *BC* (I, II, IV), *ABC* (II, III, IV)	1 each
Error	17
Total	31

Partial Confounding – Plasma Etch Experiment (Example 7.3)

The plasma etch 2^3 design from Lecture 5 (gap *A*, gas flow *B*, RF power *C*), two replicates, only four runs per shift: *ABC* confounded in replicate I, *AB* in replicate II.

```
p = pd.read_csv(URL + "plasma.txt", sep="\t")
pc = pd.concat([p.assign(Rep=1, y=p.R1),
                p.assign(Rep=2, y=p.R2)])
pc["Blk"] = np.where(pc.Rep == 1,
                     pc.A * pc.B * pc.C,    # replicate I: ABC
                     pc.A * pc.B)           # replicate II: AB
pc[["Rep", "Blk"]] = pc[["Rep", "Blk"]].astype("category")
mp = smf.ols("y ~ Rep + Rep:Blk + A*B*C", data=pc).fit()
print(anova_lm(mp).round(2))
```

```
            df     sum_sq    mean_sq       F  PR(>F)
Rep        1.0    3875.06    3875.06    1.52    0.27
Rep:Blk    2.0     458.12     229.06    0.09    0.92
A          1.0   41310.56   41310.56   16.19    0.01
B          1.0     217.56     217.56    0.09    0.78
A:B        1.0    3528.00    3528.00    1.38    0.29
C          1.0  374850.06  374850.06  146.94    0.00
A:C        1.0   94402.56   94402.56   37.01    0.00
B:C        1.0      18.06      18.06    0.01    0.94
A:B:C      1.0       6.13       6.13    0.00    0.96
Residual   5.0   12754.81    2550.96     NaN     NaN
```

Partial Confounding – Interpretation

- The sequential (type I) ANOVA with the blocks entered first does the bookkeeping for us:
  - $SS_{AB} = 3528.0$ comes from replicate I only, $[(1) + ab + c + abc - a - b - ac - bc]^2/8$,
  - $SS_{ABC} = 6.125$ comes from replicate II only, $[a + b + c + abc - ab - ac - bc - (1)]^2/8$,
  - $SS_{\text{Blocks within replicates}} = 458.125 = SS_{ABC}(\text{I}) + SS_{AB}(\text{II})$, $SS_{Rep} = 3875.06$.
- A, C and AC are important – the same conclusion as in Lecture 5.
- The error has only 5 df: 15 total -1 (replicates) -2 (blocks) -7 (effects).
- Montgomery's Table 7.11 prints $SS_C = 374{,}850.56$, $SS_{AC} = 94{,}404.56$ and $SS_E = 12{,}752.31$; the values above (and in Chapter 6) are correct, the F-ratios agree.

Blocking – Summary

- A replicated 2^k design: each replicate is a block, analysis as an RCBD.
- Block smaller than one replicate: the 2^k design is confounded in 2^p blocks; choose p independent high-order interactions, their generalized interactions are confounded too. Use Table 7.9 or check the generalized interactions yourself.
- In a single replicate the effects confounded with blocks cannot be estimated (with partial confounding they are estimated from the replicates where they are not confounded); all other effects are estimated as if no blocking had occurred.
- Replicated confounded designs: confound a different effect in each replicate (partial confounding).

Blocking – the general rules again

- Block what you can, randomize what you cannot.
- When in doubt, block.
- Block out the nuisance variables you know about, randomize as much as possible and rely on randomization to balance out the unknown nuisance effects.

Today's Exercise & Next Lecture

Ex07 – `code/01NAEX_Ex07.ipynb` (Python, Google Colab):

- The lecture examples in Python: optimal designs and prediction variance, center points and the curvature test (filtration rate), blocking a replicated design (Example 7.1), constructing blocks with the defining contrasts, filtration rate in two blocks and without blocking (Example 7.2, Section 7.5), four blocks, partial confounding (Example 7.3).
- Assignment: Problems **6.33, the center points of 6.31, 6.26 with 6.27, 7.7, 7.8 with 7.9 and 7.24** (Montgomery, 8th ed.).

Assignment: start in class, finish at home as homework. Submit `HW/01NAEX_Ex07_HW_<Surname>.ipynb` by pull request. **Deadline: Sunday 15 November 2026.**

Next lecture (23 November; no lecture on 16 November): two-level fractional factorial designs (Montgomery, Chapter 8).